



S.P.M College, Udantpuri

Bachelor Of Computer Application (BCA)

Part -1

– Hira Kumar

2. Relational /Comparison Operators (6)

- Relational operators are used for checking conditions whether the given condition is true or false. If the condition is true, it will return non-zero value, if the condition is false, it will return 0.
- Example :- Suppose we have, `int X = 5, Y = 2;` then –

Operator	Name	Description	Example
>	Greater than	Check whether the left operand is greater than right operand or not.	(X > Y) will return true
<	Smaller than	Check whether the left operand is smaller than right operand or not.	(X < Y) will return false
>=	Greater than or Equal to	Check whether the left operand is greater or equal to right operand or not.	(X >= Y) will return true
<=	Smaller than or Equal to	Check whether the left operand is smaller or equal to right operand or not.	(X <= Y) will return false
==	Equal to	Check whether the both operands are equal or not.	(X == Y) will return false
!=	Not Equal to	Check whether the both operands are equal or not.	(X != Y) will return true

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int main()
```

```
{
```

```
    int a=10, b=20;
```

```
    // true = 1, false = 0
```

```
    printf("what %d < %d = %d \n",a,b,a<b);
```

```
    printf("what %d > %d = %d \n",a,b,a>b);
```

```
    printf("what %d == %d = %d \n",a,b,a==b);
```

```
    printf("what %d <= %d = %d \n",a,b,a<=b);
```

```
    printf("what %d >= %d = %d \n",a,b,a>=b);
```

```
    printf("what %d != %d = %d \n",a,b,a!=b);
```

```
    getch();
```

```
    return 0;
```

```
}
```

C:\Users\hira\Desktop\hello.exe

what 10 < 20 = 1

what 10 > 20 = 0

what 10 == 20 = 0

what 10 <= 20 = 1

what 10 >= 20 = 0

what 10 != 20 = 1

Line Message

=== Build file: "no t

=== Build finished: 0

3. Logical Operators (5)

- Logical operators perform logical operations on Boolean expressions (expressions that evaluate to either true or false).
- There are three logical operators in C:

Suppose we have,

int X = 5, Y = 2;

Operator	Name	Description	Example
&&	AND	Return true if all conditions are true, return false if any of the condition is false.	if(X > Y && Y < X) will return true
	OR	Return false if all conditions are false, return true if any of the condition is true.	if(X > Y X < Y) will return true
!	NOT	Return true if condition is false, return false if condition is true.	if(!(X > Y)) will return false

True & False

AND (&&) Table:		
A	B	A && B
T	T	T
T	F	F
F	T	F
F	F	F

OR () Table:		
A	B	A B
T	T	T
T	F	T
F	T	T
F	F	F

NOT (!) Table:	
A	!A
T	F
F	T

1 & 0

AND (&&) Table:		
A	B	A && B
1	1	1
1	0	0
0	1	0
0	0	0

OR () Table:		
A	B	A B
1	1	1
1	0	1
0	1	1
0	0	0

NOT (!) Table:	
A	!A
1	0
0	1

Operator	Name	Example
&&	Logical And	5 < 6 && 6 < 6 // gives 0
	Logical Or	5 < 6 6 < 5 // gives 1
!	Logical Negation (Not)	!(5 == 5) // gives 0

```

#include<stdio.h>
#include<conio.h>
int main()
{
    int a=10, b=20 ,c=30;
    // true = 1, false = 0
    //AND
    printf("%d > %d && %d > %d = %d \n",a,b,a,c,a>b&&a>c);
    printf("%d > %d && %d < %d = %d\n\n",a,b,a,c,a>b&&a<c);
    //OR
    printf("%d > %d || %d > %d = %d \n",a,b,a,c,a>b||a>c);
    printf("%d > %d || %d < %d = %d\n\n",a,b,a,c,a>b||a<c);
    //NOT
    printf("%d == %d = %d",a,b,a!=b);
    getch();
    return 0;
}

```

```

C:\Users\hira\Desktop\hello.exe
10 > 20 && 10 > 30 = 0
10 > 20 && 10 < 30 = 0
10 > 20 || 10 > 30 = 0
10 > 20 || 10 < 30 = 1
10 == 20 = 1

```

OR Use bool Data type

```

#include<stdio.h>
#include<conio.h>
#include<stdbool.h> // for bool data type
int main()
{
    // true = 1, false = 0
    int a=10, b=20 ,c=30;
    bool p= a>b && b<c;
    printf("%d > %d && %d < %d = %d",a,b,b,c,p);

    getch();
    return 0;
}

```

```

C:\Users\hira\Desktop\hello.exe
10 > 20 && 20 < 30 = 0

```

Precedence & Associativity Rule of Logical Operators

Precedence

1. Partentheses
2. Uniary operator
3. Multiplication, division, and modulus
4. Addition & Subtraction
5. Relation Operator
6. Equal Operator
7. Logical Operator

Example –

Q1. Int a=5;
 Int x = 5*(a++ +2)
 Printf(“%d”,x);

Answer = 35

Q2. Int a=5,b=2;
 Int x = ++a * (a-b);
 Printf(“%d”,x);

Answer = 24

Rule – parentheses and Uniary operator यदि दोनों expression में आ जाए तो पहले parenthese करेंगे यदि उस parenthese के अंदर uniary operator हो तो जैसे (a++ +2) यदि नहीं हो तो, सबसे पहले uniary operator को solve करे फिर parenthese को solve करें |)

Q3. Int a=5, b=3, c=2, d=4;
 Int result = ++a * b / c % d + (a-b) == c && b || !c;

```
Printf("%d", result);
```

Answer = 0

Logical Operator Precedence

- जब expression में `&&` (and) या `||` (or) एक बार से ज्यादा आये तो Left to Right Solve करे, लेकिन
- यदि condition जब short-circuit हो तो - उसमें सबसे पहले short-circuit apply होगा -
- Short-circuit ऐसा condition होता है जिसमें expression solve करते समय ही पता चल जाये की इसका answer क्या होगा |

Case 1 (with short-circuit)

(यदि पहले ही true (1) हो जाए तो आगे evaluate नहीं होगा)

Q4. `5 || 3 && 4`

Answer = 1 (यहाँ `3 && 4` evaluate नहीं होगा |)

(यदि पहले ही false(0) हो जाए तो आगे evaluate नहीं होगा)

Q5. `0 && (5 || 6)`

Answer = 0 (यहाँ `5 && 6` evaluate नहीं होगा |)

Case 2 (without short-circuit)

Q6. `0 || 3 && 4` (अब यहाँ पर precedence काम करेगा)

Answer = 1

Associativity

Left to right

Q6. 0 && 5 && 7 && 9

Answer = 0

(0 && 5 → 0 , compiler यही रुक जायेगा आगे evaluate नहीं करेगा)

=====HIRA KUMAR=====